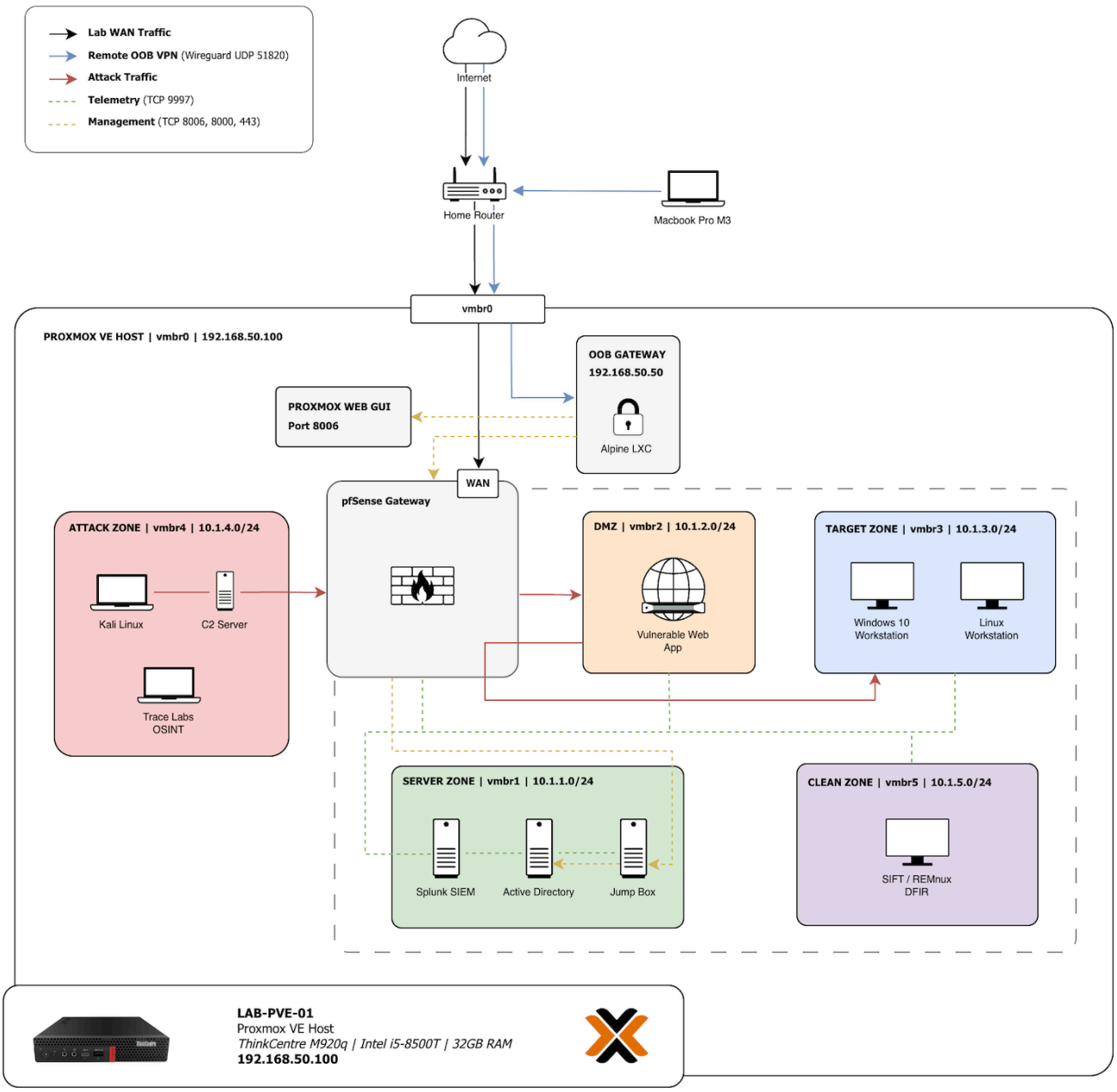
Project 1: Building a Secure Home Cyber Lab: pfSense, Active Directory, and Splunk

System Architecture & End-Goal Topology



Asset Inventory

Hostname	OS	IP Address	Subnet / Zone	Role
pfSense	FreeBSD	10.1.x.1	Gateway Core	Central Router & Policy Firewall

Hostname	OS	IP Address	Subnet / Zone	Role
oob-gateway	Alpine Linux	192.168.50.50	Out-of-Band	WireGuard LXC remote access
splunk-siem	Ubuntu Server	10.1.1.10	Server Vault (LAN)	Central Log Indexer & Analytics
dc01	Windows Server 2022	10.1.1.20	Server Vault (LAN)	Active Directory Domain Controller (Telemetry: Sysmon + Splunk Forwarder)
jumpbox-01	Windows 10	10.1.1.50	Server Vault (LAN)	Hardened Administration Workstation
vuln-web	Ubuntu Server	10.1.2.10	DMZ (OPT1)	Vulnerable Web Application
win10-client01	Windows 10	10.1.3.100	Target Zone (OPT2)	Domain-joined Active Directory client
nix-client01	Ubuntu Desktop	10.1.3.101	Target Zone (OPT2)	Linux Workstation (Telemetry: Splunk Forwarder)
kali-attack	Kali Linux	10.1.4.100	Attack Zone (OPT3)	Primary Adversary Emulation Workstation
c2-server	Ubuntu Server	10.1.4.150	Attack Zone (OPT3)	Command and Control (C2) Server
osint	Debian (Trace Labs OSINT)	10.1.4.200	Attack Zone (OPT3)	Target Profiling & OSINT Workstation
dfir-remnux	REMNux	10.1.5.100	Clean Zone (OPT4)	Forensic Investigation & Malware Analysis Workstation

Network Segmentation Policy

Interface	Subnet	Gateway	DHCP	Outbound Rule Policy
LAN (Server)	`10.1.1.0/24`	`10.1.1.1`	Disabled	Isolated. Custom pass rules for AD/DNS sync only.
OPT1 (DMZ)	`10.1.2.0/24`	`10.1.2.1`	Enabled	Internet allowed; all local lab zones blocked.
OPT2 (Target)	`10.1.3.0/24`	`10.1.3.1`	Enabled	Internet allowed; all local lab zones blocked.
OPT3 (Attack)	`10.1.4.0/24`	`10.1.4.1`	Enabled	Internet allowed; all local lab zones blocked.
OPT4 (Clean)	`10.1.5.0/24`	`10.1.5.1`	Disabled	No internet access allowed.

Module 1: Infrastructure and Isolation

Hypervisor Setup & Proxmox Storage Architecture

I set up my lab on a small Lenovo ThinkCentre M920q mini-PC. First, I turned on virtualization in the BIOS, installed Proxmox VE, and swapped the setup over to the free community repository to get rid of the license pop-up.



To get extra space, I plugged in a 1TB Samsung T7 external USB SSD. I formatted it as ext4 because I read it's much better at saving data if the USB cord gets bumped. I grabbed the drive's UUID using `blkid` and added it to my fstab file with the `nofail` flag so Proxmox wouldn't crash on startup if the drive wasn't plugged in.

ID ↑	Type	Content	Path/Target	Shared	Enabled
local	Directory	Backup, Import, ISO image, Container t…	/var/lib/vz	No	Yes
local-lvm	LVM-Thin	Disk image, Container		No	Yes
samsung-t7	Directory	Backup, Disk image, ISO image, Contai…	/mnt/samsung_t7	No	Yes

I split up my storage paths to protect my data. Critical core infrastructure like pfSense and the Active Directory domain controller were assigned to the internal drive. I chose this because I read that Active Directory databases can corrupt instantly if a USB drive gets disconnected for even a split second. Secondary workloads, heavy log indexers like Splunk, and general workstations were assigned to the external SSD to save internal drive space.

Virtual Switch Creation & Central Routing (pfSense)

Next, I used Proxmox's virtual switch settings to create five virtual networks named `vmbr1` through `vmbr5`. I left these networks completely disconnected from my physical home network so no traffic could leak out.

In the center, I installed pfSense to act as my central router and firewall, mapping my bridge numbers directly to my subnets. I assigned static IPs to my core servers and set up DHCP for the client zones.

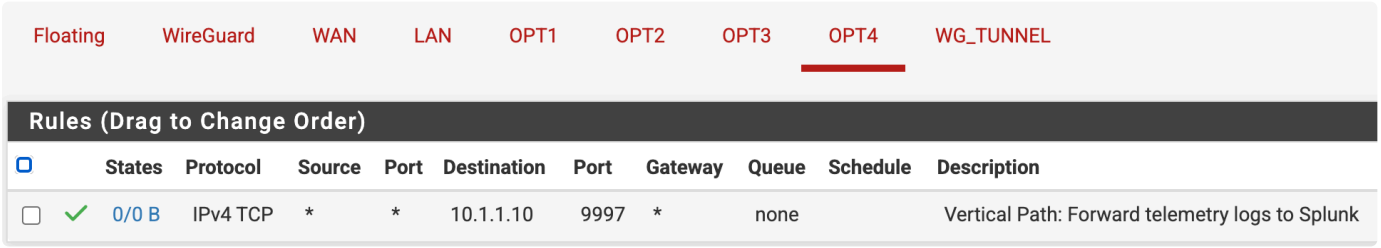
To make sure malware couldn't crawl into other networks, I made a pfSense list called `All_Private_IPs` covering my local subnets. On my main networks, I put a simple two-rule security setup at the top of the list:

1. Pass logs up to my Splunk server (10.1.1.10) on port 9997
2. Allow internet access while blocking everything on my local IP list.

This let my VMs fetch updates but stopped them from talking to each other.

		States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description
☐	✓	0/0 B	IPv4 TCP	*	*	10.1.1.10	9997	*	none		Forward telemetry logs to Splunk
☐	✓	0/0 B	IPv4 *	*	*	! All_Private_IPs	*	*	none		Allow internet, block all private networks

I set up DHCP on OPT1, OPT2, and OPT3, but kept it off on the server zone (LAN) and the OPT4 malware clean room to keep things stable. On the malware clean room (OPT4) interface, I took away the internet rule completely. Leaving only the Splunk port open meant active malware would stay completely offline but still send logs back to my SIEM.



Out-of-Band Access Gateway (Alpine WireGuard LXC)

To manage the lab remotely without exposing it to the web, I created an unprivileged Alpine Linux container running WireGuard. Because of the safety locks on unprivileged containers, it couldn't talk to the network cards. I found out I had to manually edit the container's config file on Proxmox to pass through the `/dev/net/tun` device.

```
root@pve01:~# cat /etc/pve/lxc/101.conf
arch: amd64
cores: 1
features: nesting=1
hostname: OOB-Gateway
memory: 512
net0: name=eth0,bridge=vmbr0,gw=192.168.50.1,hwaddr=BC:24:11:14:E3:C7,ip=192.168.50.50/24,type=veth
onboot: 1
ostype: alpine
rootfs: local-lvm:vm-101-disk-0,size=4G
swap: 512
unprivileged: 1
lxc.cgroup2.devices.allow: c 10:200 rwm
lxc.mount.entry: /dev/net dev/net none bind,create=dir
```

By keeping the container unprivileged and only passing through this specific device, I ensured that even if an attacker somehow compromises my WireGuard VPN, they are completely trapped inside the container and cannot break out into my physical Proxmox hypervisor.

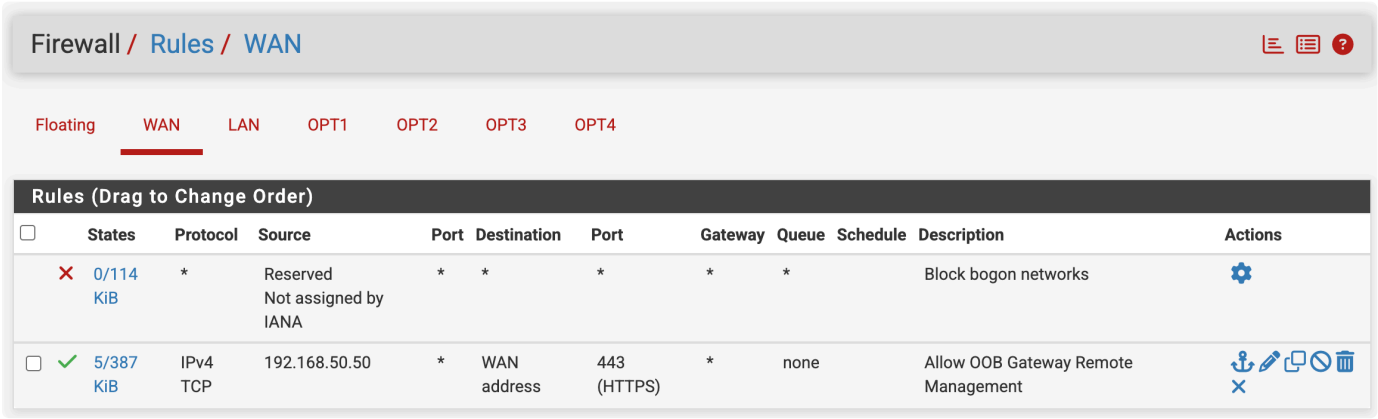
Once that was done, I forwarded UDP port 51820 on my home router, set up a split-tunnel VPN, and turned on NAT routing using `iptables` inside Alpine. At first, I couldn't reach the internal network from my laptop because the WireGuard container was missing a route back to my internal lab subnets. Adding `up ip route add 10.1.0.0/16 via 192.168.50.42` in `/etc/network/interfaces` fixed the issue.

```
OOB-Gateway:~# cat /etc/network/interfaces
auto lo
iface lo inet loopback
iface lo inet6 loopback

auto eth0
iface eth0 inet static
        address 192.168.50.50/24
        gateway 192.168.50.1
        hostname $(hostname)

up ip route add 10.1.0.0/16 via 192.168.50.42
```

Finally, I went into pfSense and added a WAN firewall rule that only allows HTTPS management access from my WireGuard IP at `192.168.50.50`.



Module 2: Core Enterprise Vault Services

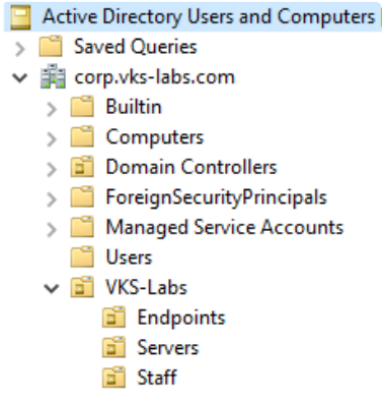
Active Directory Domain Controller (`dc01`)

I built a Windows Server 2022 domain controller at `10.1.1.20` in the Server Vault (LAN). I used standard drive settings so Windows could read the disk, mapped the network card to the VirtIO driver, and turned off the hypervisor firewall so pfSense would handle the security.

Since Windows doesn't have VirtIO drivers built-in, it booted completely offline. I mounted the guest driver ISO (`virtio-win.iso`) to the virtual CD drive and used Device Manager to install the network card driver.

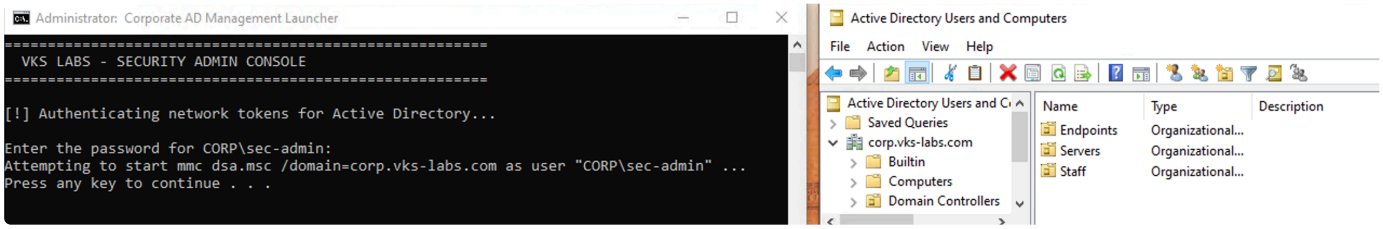


I changed the server name to `dc01` and promoted it to a domain controller for `corp.vks-labs.com` . I also set up an Organizational Unit (OU) folder structure inside Active Directory containing `Endpoints` , `Servers` , and `Staff` folders.



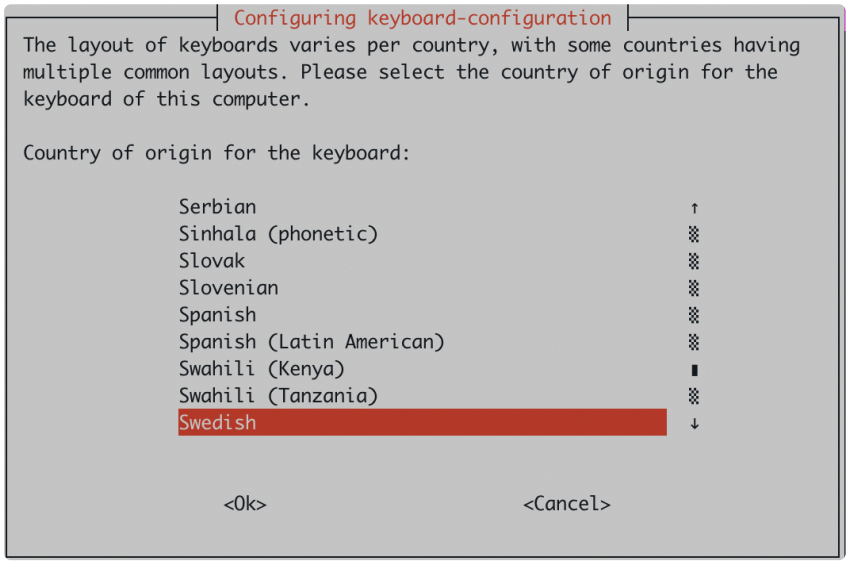
Hardened Administration Jump Box (`jumpbox-01`)

I configured a Windows Jump Box at `10.1.1.50` but left it unjoined from the domain. This ensures Domain Admin passwords are never saved in its memory where an attacker's tool could find them. I used a custom script to securely manage Active Directory from this unjoined workstation and used remote desktop on the DC for Group Policy edits.



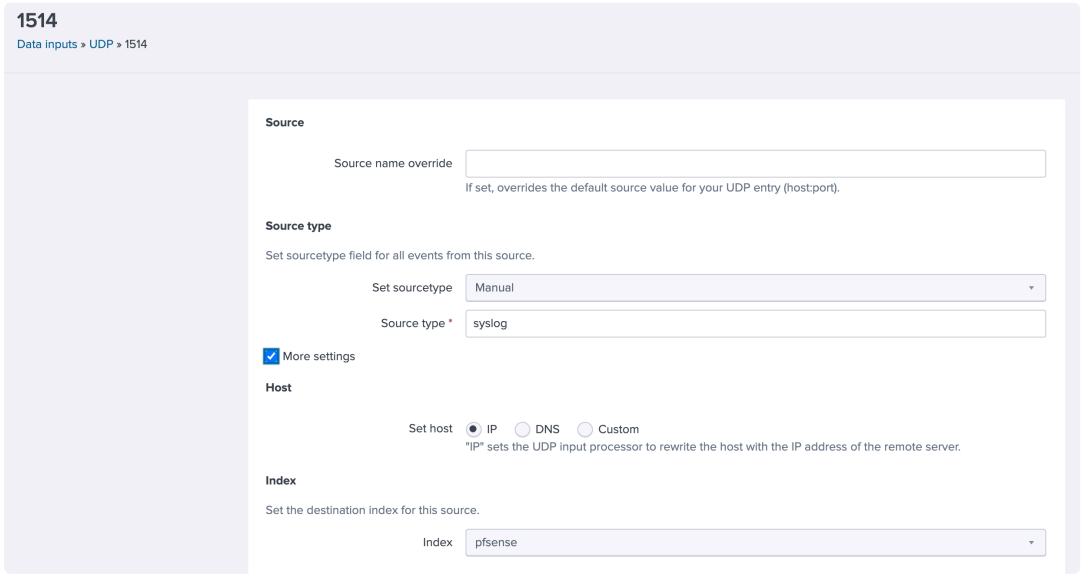
Central Log Indexer (`splunk-siem`)

I installed my main Splunk SIEM server on a fresh Ubuntu Server instance at `10.1.1.10` inside the Server Vault. Right away, I couldn't log into the console because the Ubuntu installer had defaulted to a US keyboard layout, which scrambled the special characters on my physical Swedish keyboard. I copy-pasted the password over an active SSH session to get in, then fixed the system keyboard settings.



When I tried starting Splunk, it crashed immediately because extracting the files with root privileges left the folders owned by root. A quick `chown` fixed the folder permissions.

Inside Splunk, I created a firewall index capped at 100 GB, along with dedicated Windows and Linux indexes capped at 50 GB each. Since Linux blocks regular accounts from using ports below 1024, I set up my Splunk syslog listener on port 1514 instead.



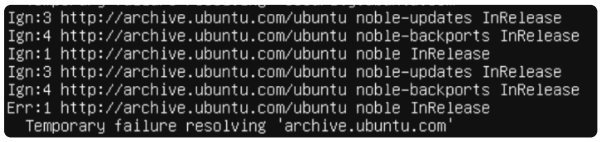
In pfSense, I turned on remote logging and pointed it to `10.1.1.10:1514`, which instantly filled my dashboard with firewall events.



Module 3: Target Environments & Telemetry Onboarding

DMZ Vulnerable Web Application (`vuln-web`)

I spun up an Ubuntu Server VM named `vuln-web` at `10.1.2.10` on my DMZ network (`vmbr2`), expanding the root logical volume (`ubuntu-lv`) to claim the full disk space. Near the end of installation, the setup hung on security updates due to DNS resolution failures for `archive.ubuntu.com`. I realized pfSense was blocking DMZ traffic from reaching my Active Directory DNS server at `10.1.1.20` on the LAN, so I skipped the updates and rebooted into the OS.



To fix the network permanently, I edited `/etc/netplan/50-cloud-init.yaml` to swap DHCP for a static IP (`10.1.2.10/24`), setting gateway `10.1.2.1` and pointing nameservers directly to `10.1.2.1` and `1.1.1.1`. Running `sudo netplan apply` restored internet access.

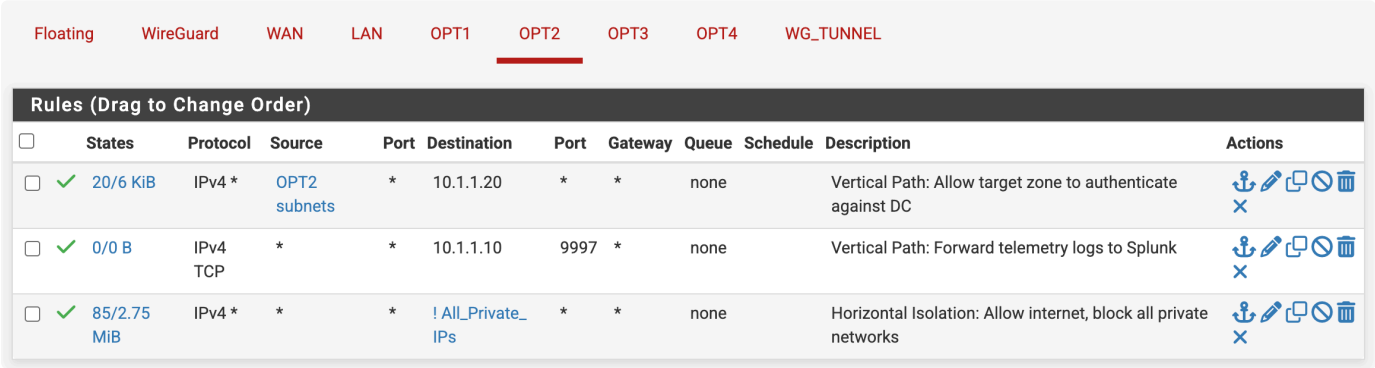


After reconnecting to `10.1.2.10` over WireGuard, I installed `qemu-guest-agent` for Proxmox reporting, installed Docker, and launched the DVWA container (`vulnerables/web-dvwa`) on port 80.

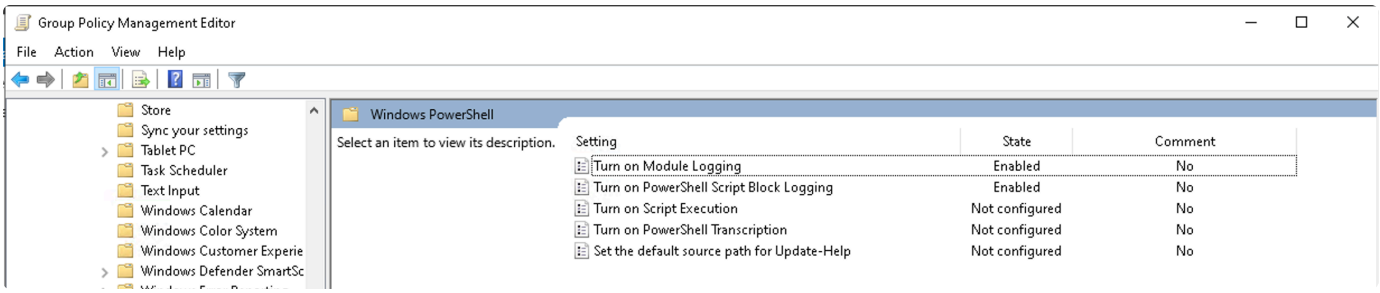


Target Endpoint Onboarding & Troubleshooting (`win10-client01`)

I deployed a Windows 10 client (`win10-client01`) at `10.1.3.100` on OPT2. When I tried to join it to the domain, the connection timed out. I traced this to my firewall isolation rule blocking local traffic across subnets. Dragging an AD authentication pass rule to the top of my OPT2 firewall rule list allowed the endpoint to reach `10.1.1.20` and resolved the timeout.



Next, I created a domain Group Policy to enable PowerShell Script Block Logging and Module Logging.



However, the client initially failed to fetch this policy. I found out there was a 9-hour time drift between my host's motherboard clock and the DC. Because Active Directory rejects any clock difference over 5 minutes, I forced the client to sync with the DC using `net time \\10.1.1.20 /set /y` and fixed the server's timezone.

```
PS C:\Windows\system32> net time \\10.1.1.20 /set /y
Current time at \\10.1.1.20 is 2026-07-06 19:03:06

The command completed successfully.

PS C:\Windows\system32>
PS C:\Windows\system32> gpupdate /force
Updating policy...

Computer Policy update has completed successfully.
User Policy update has completed successfully.
```

Even with the time aligned, test commands wouldn't show up in Splunk. A policy report showed that the client computer object was sitting in the default `Computers` folder, which doesn't support policy inheritance. I dragged the machine object into my `Endpoints` OU, renamed the computer to `win10-client01`, and rebooted.

Endpoint Telemetry Pipeline (Sysmon, GPO, & Forwarders)

To send Windows events to Splunk, I installed the Splunk Universal Forwarder on `win10-client01`. Because default Windows logs are noisy, I installed Microsoft Sysmon and applied Olaf Hartong's configuration to filter out background activity and capture process creation and network connections. I then opened the forwarder's `inputs.conf` file and added paths for Application, Security, System, Sysmon (`Microsoft-Windows-Sysmon/Operational`), and PowerShell (`Microsoft-Windows-PowerShell/Operational`) logs.

inputs - Notepad

File Edit Format View Help

```
[WinEventLog://Application]
index = windows
disabled = false

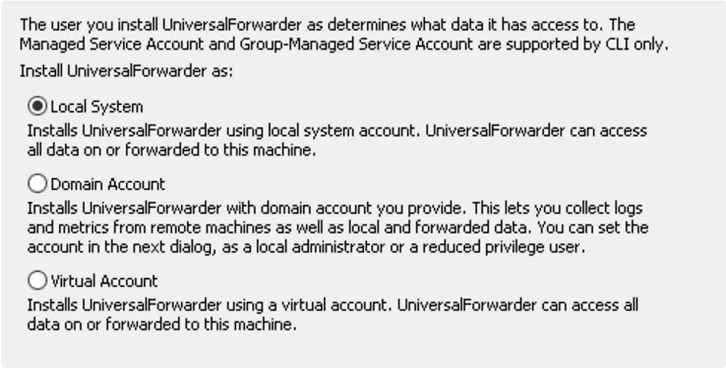
[WinEventLog://Security]
index = windows
disabled = false

[WinEventLog://System]
index = windows
disabled = false

[WinEventLog://Microsoft-Windows-Sysmon/Operational]
index = windows
disabled = false
renderXml = true
sourcetype = XmlWinEventLog:Microsoft-Windows-Sysmon/Operational


[WinEventLog://Microsoft-Windows-PowerShell/Operational]
index = windows
disabled = false
renderXml = true
```

Next, I set up telemetry on my domain controller (`dc01`). I installed Sysmon64 using Olaf Hartong's config. When I ran the Splunk Universal Forwarder installer on the DC, it kept rolling back and crashing. I found out that because a domain controller has no local user database, the default "Virtual Account" installer setting fails when it tries to add itself to local performance monitoring groups. Changing the installation service account to "Local System" resolved the issue instantly, allowing the install to finish and granting the forwarder the elevated permissions it needed to read restricted directory logs.



Once DC logs started flowing, I noticed the domain controller was generating excessive event volume from Windows Filtering Platform connections and file handles, threatening my free daily 500 MB Splunk license limit. To fix this, I edited `inputs.conf` on the DC and added a blacklist rule:

```
[WinEventLog://Security]
index = windows
disabled = false
blacklist1 = EventCode="(5156|5158|4656|4658|4690|4907)"
```

Restarting the forwarder dropped repetitive background events while preserving critical logon and policy modification logs.

Linux Endpoint Hardening (`nix-client01`)

On my Linux target (`10.1.3.101`), I noticed the Splunk forwarder was running as `root` . I reconfigured the service to run under a restricted `splunkfwd` service account. To allow Splunk to ingest bash command history without opening private user folders to everyone, I used Access Control Lists (`setfacl`) to grant read permissions strictly to `splunkfwd` :

```
sudo setfacl -m u:splunkfwd:r /home/nix-localadmin/.bash_history
```

This allowed command logs to stream safely into the `linux` index while keeping the endpoint properly hardened.

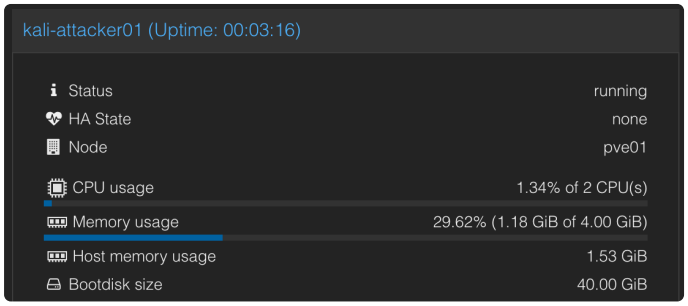
```
[nix-localadmin@nix-client01:~$ ps aux | grep splunkd                                                    ]
splunkf+   26723  0.5  8.4 370344 142912 ?       Ssl  11:32   0:00 splunkd --under-systemd --systemd-delegate=no -p 8089
 _internal_launch_under_systemd
splunkf+   26778  0.0  0.8 169776 14948 ?        Ss   11:32   0:00 [splunkd pid=26723] splunkd --under-systemd --systemd
-delegate=no -p 8089 _internal_launch_under_systemd [process-runner]
nix-loc+   26843  0.0  0.1  18408  2464 pts/0    S+   11:34   0:00 grep --color=auto splunkd
```

Module 4: Adversary Emulation & Security Validation

Attack Infrastructure Deployment

1. Primary Attack Workstation (`kali-attacker01`)

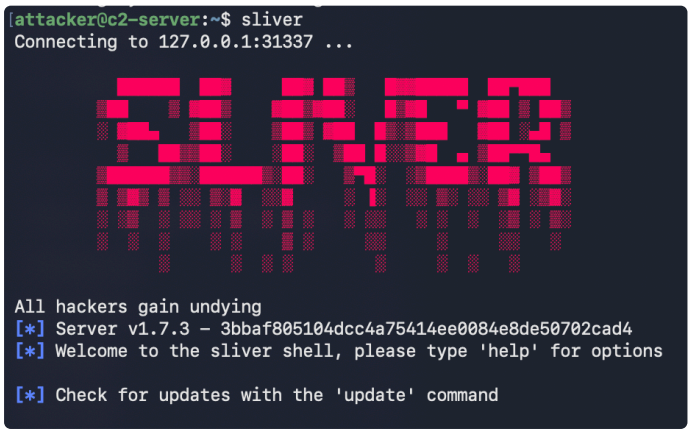
I installed my primary Kali Linux machine at `10.1.4.100` on the Attack Zone (`vmbr4`) to handle active adversary testing. I also enabled SSH and installed `qemu-guest-agent` .



2. Command and Control Server (`c2-server`)

I set up an Ubuntu Server VM at `10.1.4.150` on `vmbr4` . During installation, I set a static IP (`10.1.4.150/24`), enabled SSH, and expanded the disk to use the full 30 GB space. Once logged in, I installed `qemu-guest-agent` , updated the OS, installed `mingw-w64` for cross-compiling Windows payloads, and installed the Sliver C2 framework.

```
[attacker@c2-server:~$ sudo systemctl status qemu-guest-agent
● qemu-guest-agent.service - QEMU Guest Agent
     Loaded: loaded (/usr/lib/systemd/system/qemu-guest-agent.service; static)
     Active: active (running) since Tue 2026-07-28 12:59:27 UTC; 20s ago
   Main PID: 1419 (qemu-ga)
      Tasks: 2 (limit: 2263)
     Memory: 528.0K (peak: 932.0K)
        CPU: 12ms
     CGroup: /system.slice/qemu-guest-agent.service
             └─1419 /usr/sbin/qemu-ga
```

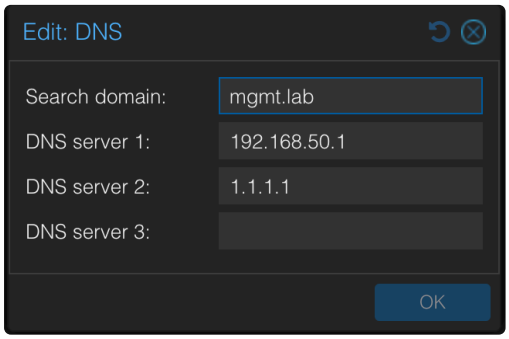


3. Target Profiling Workstation (`osint`)

I deployed the Trace Labs OSINT VM at `10.1.4.200` on `vmbr4` to keep target profiling separated from my active attack traffic. While downloading the OVA file, Proxmox failed to resolve `github.com`.

```
root@pve01:/var/lib/vz/dump# wget https://github.com/tracelabs/tlosint-vm/releases/download/2026.07/tl-osint-2026.7-vmware-amd64.ova
--2026-07-28 15:04:52--  https://github.com/tracelabs/tlosint-vm/releases/download/2026.07/tl-osint-2026.7-vmware-amd64.ova
Resolving github.com (github.com)... failed: Temporary failure in name resolution.
wget: unable to resolve host address ‘github.com’
root@pve01:/var/lib/vz/dump#
```

I updated the host's DNS settings in the Proxmox GUI to include Cloudflare (`1.1.1.1`) as a secondary DNS server.



After downloading the archive, I unpacked the contents, created a blank VM shell with no attached storage and imported the extracted disk directly into my Proxmox storage pool:

```
qm importdisk 110 tl-osint-2026.7-vmware-amd64-disk1.vmdk samsung-t7
```

I attached the imported disk as `scsi0`, set it as the boot device, and started the VM.

Since `openssh-server` wasn't included by default, I opened the console, installed SSH, enabled the service, changed the default user password, and installed `qemu-guest-agent`.

```
osint@tlosint:~$ sudo systemctl enable --now ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
```

4. Forensics & Malware Clean Room (`dfir-remnux`)

I deployed a REMnux VM (`VM 111`) at `10.1.5.100` on the OPT4 Clean Zone (`vmbr5`) to create an isolated space for reverse-engineering and malware analysis.

When I first tried to start the SSH service (`sudo systemctl enable --now ssh`), it failed. I found out that fresh REMnux builds clear out SSH host keys by default so every deployment gets unique keys. I fixed this by generating new keys with `sudo ssh-keygen -A` which let SSH start up cleanly.

```
remnux@remnux:~$ sudo ssh-keygen -A
remnux@remnux:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-07-31 07:33:54 UTC; 44s ago
```

I also changed the keyboard layout from English to Swedish.

To set up telemetry while the VM still had internet access, I temporarily connected it to `vmbr4` to download the Splunk Universal Forwarder package. I installed the forwarder, accepted the license, set it to run on boot, and pointed it to send logs directly to my Splunk server at `10.1.1.10:9997` .

Next, I configured a static IP (`10.1.5.100/24`) with gateway `10.1.5.1` in Netplan (`/etc/netplan/01-netcfg.yaml`).

```
  GNU nano 7.2                                  /etc/netplan/01-netcfg.yaml *
network:
  version: 2
  ethernets:
    ens18:
      dhcp4: no
      addresses:
        - 10.1.5.100/24
      routes:
        - to: default
          via: 10.1.5.1
```

Netplan gave a permissions warning, so I restricted access to the file using `sudo chmod 600 /etc/netplan/01-netcfg.yaml` .

```
remnux@remnux:~$ sudo netplan apply

** (generate:2831): WARNING **: 07:52:03.518: Permissions for /etc/netplan/01-netcfg.yaml are too open. Netplan configur
ation should NOT be accessible by others.
```

```
remnux@remnux:~$ sudo chmod 600 /etc/netplan/01-netcfg.yaml
```

To clean up the old DHCP address (`10.1.4.103`), I ran `sudo ip addr flush dev ens18` before re-applying Netplan.

Finally, I switched the VM's network interface to `vmbr5` in Proxmox and enabled the QEMU Guest Agent (`qm set 111 --agent enabled=1`) to allow clean OS shutdowns, live IP reporting, and safe snapshotting.

```
remnux@remnux:~$ sudo systemctl status qemu-guest-agent
● qemu-guest-agent.service - QEMU Guest Agent
     Loaded: loaded (/usr/lib/systemd/system/qemu-guest-agent.service; static)
     Active: active (running) since Fri 2026-07-31 07:57:12 UTC; 1min 1s ago
```

Boundary Isolation & Security Validation Testing

I ran network validation tests across the Attack Zone (`vmbr4`) and the Clean Zone (`vmbr5`) to confirm my pfSense firewall rules were working properly.

1. Attack Zone Boundary Checks

From Kali (`10.1.4.100`), an initial HTTP request to the DMZ web server (`curl -I http://10.1.2.10`) timed out because my default isolation rule blocks traffic between subnets. I added a pass rule at the top of the OPT3 firewall interface to allow HTTP/HTTPS traffic to `10.1.2.0/24` .

Rules (Drag to Change Order)

	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description	Actions
☐	✓ 0/0 B	IPv4 *	OPT3 subnets	*	10.1.2.0/24	*	*	none		Allow Attack Zone to reach DMZ	
☐	✓ 0/639 B	IPv4 UDP	*	*	OPT3 address	53 (DNS)	*	none		Pass DNS queries to Gateway	
☐	✓ 0/0 B	IPv4 TCP	*	*	10.1.1.10	9997	*	none		Vertical Path: Forward telemetry logs to Splunk	
☐	✓ 0/760 B	IPv4 *	*	*	! All_Private_IPs	*	*	none		Horizontal Isolation: Allow internet, block all private networks	

Retesting `curl -I http://10.1.2.10` returned an immediate `HTTP/1.1 302 Found` response.

```
┌──(attacker㉿kali-attacker01)-[~]
└─$ curl -I http://10.1.2.10
HTTP/1.1 302 Found
Date: Wed, 22 Jul 2026 11:19:34 GMT
Server: Apache/2.4.25 (Debian)
Set-Cookie: PHPSESSID=6j9vm38j0m3a8a1r4ue371c2k2; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=6j9vm38j0m3a8a1r4ue371c2k2; path=/
Set-Cookie: security=low
Location: login.php
Content-Type: text/html; charset=UTF-8
```

Next, I tested connections from Kali to the Domain Controller (`10.1.1.20`) and Splunk server (`10.1.1.10`) in the Server Vault. Pinging `10.1.1.20` resulted in 100% packet loss, and `nc -zvw3 10.1.1.10 8000` timed out completely. This confirmed pfSense allows my attack machine to reach exposed DMZ services while blocking lateral movement into sensitive internal networks.

```
┌──(attacker㉿kali-attacker01)-[~]
└─$ ping -c 3 10.1.1.20
PING 10.1.1.20 (10.1.1.20) 56(84) bytes of data.

--- 10.1.1.20 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2051ms


┌──(attacker㉿kali-attacker01)-[~]
└─$ nc -zvw3 10.1.1.10 8000
10.1.1.10: inverse host lookup failed: Unknown host
(UNKNOWN) [10.1.1.10] 8000 (?) : Connection timed out
```

2. Clean Zone Isolation & Telemetry Egress Checks

From `dfir-remnux` (`10.1.5.100`), I ran two network checks to confirm the security of the malware clean room:

Running `ping -c 3 1.1.1.1` resulted in 100% packet loss. This confirmed outbound internet access is completely blocked on `vmbr5` , keeping detonated malware from connecting to external Command and Control (C2) servers.

```
remnux@remnux:~$ ping -c 3 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.

--- 1.1.1.1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2086ms
```

Running `nc -zvw3 10.1.1.10 9997` returned `[10.1.1.10] 9997 (?) open` . This verified that despite being cut off from the internet, the VM can still send logs to the Splunk indexer.

```
remnux@remnux:~$ nc -zvw3 10.1.1.10 9997
10.1.1.10: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [10.1.1.10] 9997 (?) open
```

Storage Format Conversion & Baseline Snapshotting

To save the clean baseline state of the lab before running attack simulations or analyzing malware, I created snapshots (`Clean-Base`) across all VMs.

When trying to snapshot `dfir-remnux` , Proxmox threw an error: *"The current guest configuration does not support taking new snapshots"*. Running `qm config 111 | grep scsi` on `pve01` showed the disk was stored as a `.raw` image (`vm-111-disk-0.raw`) on the directory storage pool (`samsung-t7`). I found out that RAW files on directory storage don't support snapshots.

```
root@pve01:~# qm config 111 | grep scsi
boot: order=scsi0
scsi0: samsung-t7:111/vm-111-disk-0.raw,iothread=1,size=100G
scsihw: virtio-scsi-single
```

I converted the disk to QCOW2 using the Proxmox CLI:

```
qm disk move 111 scsi0 samsung-t7 --format qcow2 --delete 1
```

This converted the disk to `vm-111-disk-0.qcow2` and deleted the old `.raw` file to save space.

When taking baseline snapshots across `sec-pfsense` , `sec-splunk` , `win10-client01` , `kali-attacker01` , and `dfir-remnux` , I intentionally unchecked **Include RAM**. Leaving out active memory keeps snapshot sizes small, makes rollbacks much faster, and ensures VMs boot up fresh without stuck network connections.

Project Outcomes & Next Steps

With Project 1 wrapped up, I now have a fully working, secure home cyber lab ready to go. Everything for routing, storage, Active Directory, and log collection is set up and tested:

- **Centralized Routing & Isolation:** I set up pfSense to manage five isolated virtual networks (`vmbr1` – `vmbr5`). This lets my VMs get out to the internet when needed, while keeping my REMnux malware clean room (`OPT4`) completely offline.
- **Log Collection Pipeline:** My Splunk server (`10.1.1.10`) is actively pulling in firewall logs from pfSense, Sysmon and Windows Event logs from my domain controller and client, and bash command logs from Linux endpoints.
- **Security Validation:** I verified that my attack tools on OPT3 can hit the vulnerable web server in the DMZ without reaching any of my core servers on the internal network. The malware clean room stays isolated from the web, but can still stream logs straight to Splunk on port 9997.
- **Easy Rollbacks:** Converting all my VM disks to QCOW2 means I can take quick `Clean-Base` snapshots without wasting extra drive space on RAM dumps.

Now that the lab is fully segmented, monitored, and protected with clean snapshots, I'm ready for **Project 2: Automated Adversary Emulation & SIEM Detections**. My next steps will focus on building custom Python scripts to launch automated attack payloads (web exploits and covert DNS exfiltration), automatically validating my Splunk detection rules via API, and testing post-exploitation behavior with Sliver C2 and Atomic Red Team.